A02-Vector-Databases

1 Vector Databases

Document Version: 1.0
Generated: December 4, 2025
Standard: Vector Database API Specifications
Status: Industry Standards

1.1 Overview

Vector databases are specialized systems for storing and searching high-dimensional embeddings. They enable semantic search, similarity matching, and AI-powered retrieval at scale.

1.1.1 Key Capabilities

- **ANN Search**: Approximate nearest neighbor search
- **Semantic Search**: Find similar items by meaning
- **Scalability**: Handle millions/billions of vectors
- **Filtering**: Metadata-based filtering with vectors
- **Real-time**: Fast index updates
- **Distributed**: Multi-node clusters

1.1.2 Index Structures

Type	Complexity	Accuracy	Speed	Best For
HNSW	O(log N)	95-99%	Medium	General purpose
IVF	O(k + log N)	90-95%	Fast	Large-scale
LSH	O(1)	70-90%	Very Fast	Approximate
Annoy	O(log N)	90-95%	Medium	Sparse data
ScaNN	O(log N)	95-99%	Fast	Google

1.2 Authoritative References

- **Pinecone**: https://docs.pinecone.io
- **Weaviate**: https://weaviate.io/developers/weaviate/
- **Milvus**: https://milvus.io/docs
- **Qdrant**: https://qdrant.tech/documentation
- **FAISS**: https://github.com/facebookresearch/faiss

1.3 Vector Database Specifications

1.3.1 Pinecone

API Endpoint Structure:

```
POST https://controller.{region}.pinecone.io/indexes
GET  https://controller.{region}.pinecone.io/indexes
POST https://{index_name}-{project_id}.svc.{region}.pinecone.io/vectors/upsert
POST https://{index_name}-{project_id}.svc.{region}.pinecone.io/query
```

Create Index Request:

```
{
  "name": "document-index",
  "dimension": 1536,
  "metric": "cosine",
  "spec": {
    "serverless": {
      "cloud": "aws",
      "region": "us-west-2"
    }
  }
}
```

Upsert Vectors:

```
{
  "vectors": [
    {
      "id": "doc-001",
      "values": [0.1, 0.2, 0.3, ...],
      "metadata": {
        "source": "document.pdf",
        "page": 1,
        "timestamp": "2024-01-01T00:00:00Z"
      }
    }
  ],
  "namespace": "production"
}
```

Query:

```
{
  "vector": [0.1, 0.2, 0.3, ...],
  "topK": 10,
  "includeMetadata": true,
  "namespace": "production",
  "filter": {
    "source": {"$eq": "document.pdf"}
  }
}
```

Response:

```
{
  "matches": [
    {
      "id": "doc-001",
      "score": 0.89,
      "metadata": {
        "source": "document.pdf",
        "page": 1
      }
    }
  ],
  "namespace": "production"
}
```

Python Client:

```python
from pinecone import Pinecone

pc = Pinecone(api_key="xxx")
index = pc.Index("document-index")

# Upsert
index.upsert(
    vectors=[
        ("doc-001", [0.1, 0.2, 0.3], {"source": "doc.pdf"}),
        ("doc-002", [0.2, 0.3, 0.4], {"source": "doc.pdf"}),
    ]
)

# Query
results = index.query(
    vector=[0.1, 0.2, 0.3],
    top_k=10,
    include_metadata=True,
    filter={"source": {"$eq": "doc.pdf"}}
)

for match in results.matches:
    print(f"{match.id}: {match.score}")
```

1.3.2 Weaviate

REST API Structure:

```
POST  /v1/objects                 # Create object
GET   /v1/objects/{id}            # Get object
POST  /v1/objects/search          # Search
POST  /v1/graphql                 # GraphQL query
POST  /v1/classifiers             # Classification
```

Class Schema:

```
{
  "class": "Document",
  "description": "A text document",
  "vectorIndexConfig": {
    "distance": "cosine",
    "ef": 100,
    "efConstruction": 128,
    "maxConnections": 30,
    "vectorCacheMaxObjects": 100000
  },
  "properties": [
    {
      "name": "content",
      "dataType": ["text"],
      "description": "Document content"
    },
    {
      "name": "source",
      "dataType": ["string"],
      "description": "Document source"
    },
    {
      "name": "timestamp",
      "dataType": ["date"],
      "description": "Creation time"
    }
  ]
}
```

Add Object:

```
{
  "class": "Document",
  "properties": {
    "content": "This is document content",
    "source": "document.pdf",
    "timestamp": "2024-01-01T00:00:00Z"
  }
}
```

Vector Search Query:

```
{
  Get {
    Document(
      nearVector: {
        vector: [0.1, 0.2, 0.3]
      }
      limit: 10
      where: {
        path: ["source"]
        operator: Equal
        valueString: "document.pdf"
      }
    ) {
      _additional {
        distance
        vector
      }
      content
      source
      timestamp
    }
  }
}
```

Python Client:

```python
import weaviate

client = weaviate.connect_to_local()

# Create class
class_obj = {
    "class": "Document",
    "properties": [
        {"name": "content", "dataType": ["text"]},
        {"name": "source", "dataType": ["string"]}
    ]
}
client.collections.create_from_dict(class_obj)

# Add object
doc_collection = client.collections.get("Document")
doc_collection.data.insert({
    "content": "Document text",
    "source": "doc.pdf"
})

# Vector search
response = doc_collection.query.near_vector(
    near_vector=[0.1, 0.2, 0.3],
    limit=10,
    where=weaviate.classes.query.Filter.by_property("source").equal("doc.pdf")
)

for obj in response.objects:
    print(f"Score: {obj.metadata.distance}")
    print(f"Content: {obj.properties['content']}")
```

1.3.3 Milvus

Collection Definition (Python):

```python
from pymilvus import Collection, FieldSchema, CollectionSchema, DataType

fields = [
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),
    FieldSchema(name="content", dtype=DataType.VARCHAR, max_length=65535),
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1536),
    FieldSchema(name="source", dtype=DataType.VARCHAR, max_length=256)
]

schema = CollectionSchema(fields, "Document collection")
collection = Collection("documents", schema)

# Create index
index_params = {
    "index_type": "IVF_FLAT",
    "metric_type": "L2",
    "params": {"nlist": 1024}
}
collection.create_index("embedding", index_params)
```

Insert Data:

```python
data = [
    [1, 2, 3],  # IDs
    ["doc1", "doc2", "doc3"],  # content
    [[0.1, 0.2, ..., 0.1536], ...],  # embeddings
    ["source1", "source2", "source3"]  # source
]

collection.insert(data)
collection.flush()
```

Search Query:

```python
search_params = {
    "metric_type": "L2",
    "params": {"nprobe": 10}
}

query_vector = [0.1, 0.2, ..., 0.1536]

results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    param=search_params,
    limit=10,
    expr="source == 'source1'",
    output_fields=["id", "content", "source"]
)

for hit in results[0]:
    print(f"ID: {hit.id}, Distance: {hit.distance}")
```

1.3.4 Qdrant

API Structure:

```
POST   /collections                # Create collection
POST   /collections/{name}/points   # Upsert points
POST   /collections/{name}/points/search  # Search
DELETE /collections/{name}          # Delete collection
```

Create Collection:

```
{
  "vectors": {
    "size": 1536,
    "distance": "Cosine",
    "on_disk": true
  },
  "quantization_config": {
    "scalar": {
      "type": "int8",
      "quantile": 0.99,
      "always_ram": false
    }
  }
}
```

Upsert Points:

```
{
  "points": [
    {
      "id": 1,
      "vector": [0.1, 0.2, 0.3, ...],
      "payload": {
        "source": "document.pdf",
        "page": 1,
        "timestamp": "2024-01-01T00:00:00Z"
      }
    }
  ]
}
```

Search Request:

```
{
  "vector": [0.1, 0.2, 0.3, ...],
  "limit": 10,
  "filter": {
    "must": [
      {
        "key": "source",
        "match": {
          "text": "document.pdf"
        }
      }
    ]
  }
}
```

Python Client:

```python
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = QdrantClient(":memory:")

# Create collection
client.create_collection(
    collection_name="documents",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)

# Upsert points
client.upsert(
    collection_name="documents",
    points=[
        PointStruct(
            id=1,
            vector=[0.1, 0.2, 0.3, ...],
            payload={"source": "doc.pdf", "page": 1}
        )
    ]
)

# Search
results = client.search(
    collection_name="documents",
    query_vector=[0.1, 0.2, 0.3, ...],
    limit=10,
    query_filter={
        "must": [
            {"key": "source", "match": {"text": "doc.pdf"}}
        ]
    }
)

for result in results:
    print(f"ID: {result.id}, Score: {result.score}")
```

1.3.5 Chroma

Simple Local Usage:

```python
import chromadb

# Create client
client = chromadb.Client()

# Create collection
collection = client.create_collection(
    name="documents",
    metadata={"hnsw:space": "cosine"}
)

# Add documents
collection.add(
    ids=["doc1", "doc2", "doc3"],
    documents=[
        "First document",
        "Second document",
        "Third document"
    ],
    metadatas=[
        {"source": "file1.pdf"},
        {"source": "file2.pdf"},
        {"source": "file1.pdf"}
    ]
)

# Query (embeddings computed automatically)
results = collection.query(
    query_texts=["similar content"],
    n_results=10,
    where={"source": {"$eq": "file1.pdf"}}
)

print(results)
```

1.3.6 FAISS (Facebook AI Similarity Search)

Index Construction (Python):

```python
import faiss
import numpy as np

# Data
vectors = np.random.random((100000, 1536)).astype('float32')
index = faiss.IndexFlatL2(1536)  # L2 distance

# Add vectors
index.add(vectors)

# Search
query = np.random.random((1, 1536)).astype('float32')
distances, indices = index.search(query, k=10)

print(f"Nearest neighbors: {indices[0]}")
print(f"Distances: {distances[0]}")
```

With GPU:

```python
import faiss

# Create GPU index
res = faiss.StandardGpuResources()
index_cpu = faiss.IndexFlatL2(1536)
index = faiss.index_cpu_to_gpu(res, 0, index_cpu)

# Fast GPU search
distances, indices = index.search(query, k=10)
```

1.4 Comparison Matrix

Feature	Pinecone	Weaviate	Milvus	Qdrant	Chroma
Deployment	Serverless	Self-hosted	Self-hosted	Both	Local/Cloud
Index Type	HNSW	HNSW	IVF/HNSW	HNSW	HNSW
Filtering	Native	GraphQL	SQL	Payload	Field filter
Scalability	Managed	Horizontal	Distributed	Distributed	Single node
Pricing	Usage-based	Free	Free	Free	Free
API	REST/Python	REST/GraphQL	Python/gRPC	REST/Python	Python

1.5 Procedural Use-Cases

1.5.1 Use-Case 1: Build RAG System with Pinecone

```python
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer
import os

class RAGSystem:
    def __init__(self, index_name: str):
        self.pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
        self.index = self.pc.Index(index_name)
        self.model = SentenceTransformer("all-MiniLM-L6-v2")

    def ingest_documents(self, documents: list, batch_size: int = 100):
        """Embed and store documents"""
        for i in range(0, len(documents), batch_size):
            batch = documents[i:i + batch_size]

            # Embed batch
            embeddings = self.model.encode(batch)

            # Prepare vectors
            vectors = [
                (f"doc-{i+j}", embeddings[j], {"text": batch[j]})
                for j in range(len(batch))
            ]

            # Upsert to Pinecone
            self.index.upsert(vectors=vectors)

    def retrieve_documents(self, query: str, top_k: int = 5) -> list:
        """Find relevant documents"""
        # Embed query
        query_embedding = self.model.encode(query)

        # Search
        results = self.index.query(
            vector=query_embedding,
            top_k=top_k,
            include_metadata=True
        )

        # Extract documents
        documents = [match.metadata["text"] for match in results.matches]
        return documents

# Usage
rag = RAGSystem("document-index")
docs = ["Document 1", "Document 2", ...]
rag.ingest_documents(docs)

results = rag.retrieve_documents("search query")
print(results)
```

1.5.2 Use-Case 2: Semantic Search with Weaviate

```python
import weaviate
from weaviate.classes.query import Filter

class SemanticSearch:
    def __init__(self):
        self.client = weaviate.connect_to_local()

    def setup(self):
        """Initialize Weaviate schema"""
        schema = {
            "class": "Article",
            "properties": [
                {"name": "title", "dataType": ["text"]},
                {"name": "content", "dataType": ["text"]},
                {"name": "category", "dataType": ["string"]}
            ]
        }
        self.client.collections.create_from_dict(schema)

    def search(self, query: str, category: str = None):
        """Perform semantic search"""
        articles = self.client.collections.get("Article")

        filters = None
        if category:
            filters = Filter.by_property("category").equal(category)

        response = articles.query.near_text(
            query=query,
            limit=10,
            where=filters
        )

        return response.objects

# Usage
searcher = SemanticSearch()
results = searcher.search("machine learning", category="AI")
```

1.6 Tools & Ecosystem

Tool	Purpose	URL
Embedding Models	Generate vectors	https://huggingface.co/models?task=sentence-similarity
Vector CLI	DB management	https://github.com/qdrant/vector-db-benchmark
Vanna	RAG on databases	https://github.com/vanna-ai/vanna
LangChain	Orchestration	https://docs.langchain.com/docs/integrations/vectorstores

---

Navigation: Back to Index | Previous: Vector Embeddings | Next: RAG Patterns